

이산수학

Report 1 :역행렬 구하는 프로그램

20233144 인공지능학부 변수양

GitHub: https://github.com/suyamg/discrete_mathematics

목차

1. 행렬 입력 기능
2. 행렬식을 이용한 역행렬 계산 기능
3. 가우스-조던 소거법(Gauss-Jordan elimination)을 이용한 역행렬 계산 기능
4. 결과 출력 및 비교 기능
5. 추가 기능
6. 프로그램 전체 구조
7. TEST

1. 행렬 입력 기능

- create_matrix_inputs()
→ 사용자가 입력한 n값에 맞게 Entry 위젯으로 행렬 입력 칸 생성
- entry_n (행렬 크기 입력칸)
- btn_create (Input 버튼 → create_matrix_inputs 실행)

```
entry_n = tk.Entry(top_frame, width=5, font=("Apple SD Gothic Neo", 12), justify="center", bg="white")
entry_n.pack(side="left", padx=5)
```

```
def create_matrix_inputs():
    global matrix_entries
    for widget in matrix_frame.winfo_children():
        widget.destroy()
    matrix_entries = []
    try:
        n = int(entry_n.get())
    except ValueError:
        messagebox.showerror("오류", "정수를 입력하세요.")
        return

    if n == 1:
        messagebox.showwarning("안내", "1x1 행렬은 입력칸 생성이 필요 없습니다.")
        return

    for i in range(n):
        row_entries = []
        for j in range(n):
            e = tk.Entry(matrix_frame, width=7, font=("Arial", 11), justify="center", bg="white")
            e.grid(row=i, column=j, padx=3, pady=3)
            row_entries.append(e)
        matrix_entries.append(row_entries)
```

- create_matrix_inputs() 함수에서 entry_n 값으로 크기를 받고 matrix_entries에 Entry 위젯으로 행렬 데이터를 받음

2. Adjoint 방법 (행렬식)

- `adjoint_inverse(matrix)` 함수에서 행렬식(det)을 구하고 (determinant_with_steps) Cofactor $\rightarrow \text{adj}(A) \rightarrow 1/\det(A) \cdot \text{adj}(A)$ 로 역행렬 계산
- Cofactor $\rightarrow \text{adj}(A) \rightarrow 1/\det(A) \cdot \text{adj}(A)$ 로 역행렬 계산

```
def adjoint_inverse(matrix):
    n = len(matrix)
    det, det_log = determinant_with_steps(matrix)
    if abs(det) < 1e-9:
        raise ValueError("역행렬 불가: det(A)=0 입니다.")

    logs = []
    logs.append("행렬식 det(A)\n, end=")
    logs.append(det_log)
    cof = cofactor_matrix(matrix)
    logs.append("\n여인자 행렬 (Cofactor Matrix)\n" + pretty_matrix(cof))

    adj = list(map(list, zip(*cof)))
    logs.append("\n수반행렬 adj(A) = Cofactor^T\n" + pretty_matrix(adj))

    inv = [[adj[i][j] / det for j in range(n)] for i in range(n)]
    logs.append(f"\nA^(-1) = (1/det(A)) * adj(A), det(A) = {det:.2f}\n")
    logs.append(pretty_matrix(inv))

    return inv, "\n".join(logs)
```

3. 가우스-조던 소거법(Gauss-Jordan elimination)

- gauss_jordan_inverse(matrix)

: 모든 행을 단위행렬로 바꾸는 과정 로그 기록

- calculate_inverse() 함수 내부에서 호출됨

```
def gauss_jordan_inverse(matrix):
    n = len(matrix)
    A = deepcopy(matrix)
    I = [[float(i == j) for j in range(n)] for i in range(n)]
    logs = []
    logs.append("초기 상태 \n" + pretty_two_matrices(A, I))

    for i in range(n):
        if A[i][i] == 0:
            for j in range(i+1, n):
                if A[j][i] != 0:
                    A[i], A[j] = A[j], A[i]
                    I[i], I[j] = I[j], I[i]
                    logs.append(f"\n행 {i} ↔ 행 {j} 교환 \n{pretty_two_matrices(A, I)}")
                    break
            else:
                raise ValueError("역행렬 불가: 행렬식이 0입니다.")
        pivot = A[i][i]
        for k in range(n):
            A[i][k] /= pivot
            I[i][k] /= pivot
        logs.append(f"\n행 {i}를 피벗 {pivot:.2f}로 나눔 \n{pretty_two_matrices(A, I)}")

        for j in range(n):
            if j != i:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
                    I[j][k] -= factor * I[i][k]
                logs.append(f"\n행 {j}에서 {factor:.2f} 행 {i} 빼기 \n{pretty_two_matrices(A, I)}")

    return I, "\n".join(logs)
```

- 단위행렬 I와 확장행렬 [A|I] 시작
- 피벗 선택 및 행 연산 수행
- det=0(피벗 불가) 시 ValueError 발생

4. 결과 출력

- `calculate_inverse()`: adjoint 방법과 Gauss-Jordan 방법 각각 실행하여 `show_results()`에서 두 결과를 GUI로 출력, `compare_matrices()`로 동일 여부 비교 후 결과 메시지 표시

```
def calculate_inverse():
    try:
        n = len(matrix_entries)
        if n == 0:
            messagebox.showwarning("입력 오류", "먼저 Input A를 누르세요.")
            return
        matrix = get_matrix(matrix_entries)
        det, _ = determinant_with_steps(matrix)
        if abs(det) < 1e-9:
            messagebox.showerror("역행렬 불가", "det(A) = 0 이므로 역행렬을 구할 수 없습니다.")
            return
        inv_adj, log_adj = adjoint_inverse(matrix)
        inv_jordan, log_jordan = gauss_jordan_inverse(matrix)
        show_results(inv_adj, log_adj, inv_jordan, log_jordan)
        if compare_matrices(inv_adj, inv_jordan):
            messagebox.showinfo("비교 결과", "✅ 두 방법의 결과가 동일합니다.")
        else:
            messagebox.showwarning("비교 결과", "⚠️ 두 방법의 결과가 다릅니다.")
    except Exception as e:
        messagebox.showerror("오류", str(e))
```

```
def show_results(inv_adj, log_adj, inv_jordan, log_jordan):
    for widget in result_frame.winfo_children():
        widget.destroy()
    titles = ["Adjoint Method (Determinant-based)", "Gauss-Jordan Elimination"]
    matrices = [inv_adj, inv_jordan]
    logs = [log_adj, log_jordan]

    for idx, (title, mat, log) in enumerate(zip(titles, matrices, logs)):
        frame = tk.Frame(result_frame, bg="white", bd=1, relief="solid")
        frame.grid(row=0, column=idx, padx=10, pady=10, sticky="n")
        tk.Label(frame, text=title, font=("Apple SD Gothic Neo", 10, "bold"),
                 bg="white", fg="#333").pack(pady=5)
        grid_frame = tk.Frame(frame, bg="white")
        grid_frame.pack(padx=5, pady=5)
        n = len(mat)
        for i in range(n):
            for j in range(n):
                lbl = tk.Label(grid_frame, text=f"{mat[i][j]:.2f}",
                              font=("Apple SD Gothic Neo", 12),
                              width=8, borderwidth=1, relief="ridge",
                              bg=█ "#f5f5f5")
                lbl.grid(row=i, column=j, padx=1, pady=1)
        log_box = tk.Text(frame, height=18, width=55,
                          wrap="word", font=("Courier New", 10),
                          bg=█ "#f5f5f5", fg="black",
                          insertbackground=█ "#1e3a5f",
                          highlightthickness=0, borderwidth=0)
        log_box.pack(pady=5)
        log_box.tag_config("explain", foreground="black")
        log_box.tag_config("matrix", foreground=█ "#1565c0")
        for line in log.split("\n"):
            if "[" in line or "]" in line:
                log_box.insert("end", line + "\n", "matrix")
            else:
                log_box.insert("end", line + "\n", "explain")
        log_box.config(state="disabled")
```

4. 역행렬 결과 비교

- `compare_matrices(m1, m2, tol=1e-9)`: 두 방법의 결과가 동일한지 비교

```
def compare_matrices(m1, m2, tol=1e-9):  
    n = len(m1)  
    for i in range(n):  
        for j in range(n):  
            if abs(m1[i][j] - m2[i][j]) > tol:  
                return False  
    return True
```

```
if compare_matrices(inv_adj, inv_jordan):  
    messagebox.showinfo("비교 결과", "✅ 두 방법의 결과가 동일합니다.")  
else:  
    messagebox.showwarning("비교 결과", "⚠️ 두 방법의 결과가 다릅니다.")  
except Exception as e:  
    messagebox.showerror("오류", str(e))
```

5. 추가 기능: 행렬식 계산 버튼

- `determinant_with_steps(matrix, depth=0)`: 행렬식(det) 계산을 단계별 로그와 함께 수행

```
def pretty_matrix(M):
    n = len(M)
    lines = []
    for i in range(n):
        if i == 0:
            line = "[" + " ".join(f"{x:6.2f}" for x in M[i]) + " ]"
        elif i == n-1:
            line = "]" + " ".join(f"{x:6.2f}" for x in M[i]) + " ]"
        else:
            line = "| " + " ".join(f"{x:6.2f}" for x in M[i]) + " |"
        lines.append(line)
    return "\n".join(lines)
```

```
def determinant_with_steps(matrix, depth=0):
    n = len(matrix)
    indent = " " * depth
    if n == 1:
        return matrix[0][0], f"{indent}det(\n{pretty_matrix(matrix)}\n) = {matrix[0][0]:.2f}\n"
    if n == 2:
        val = matrix[0][0]*matrix[1][1] - matrix[0][1]*matrix[1][0]
        log = (f"{indent}det(\n{pretty_matrix(matrix)}\n{indent})\n"
            f"{indent}= {matrix[0][0]} × {matrix[1][1]} - {matrix[0][1]} × {matrix[1][0]} "
            f"= {val:.2f}\n")
        return val, log

    det_val = 0
    log = f"{indent}det({n} × {n} 행렬)\n{pretty_matrix(matrix)}\n"

    superscripts = {0:"0", 1:"1", 2:"2", 3:"3", 4:"4",
                    5:"5", 6:"6", 7:"7", 8:"8", 9:"9"}
    for c in range(n):
        minor = [row[:c] + row[c+1:] for row in matrix[1:]]
        sub_val, sub_log = determinant_with_steps(minor, depth+1)
        term = ((-1)**c) * matrix[0][c] * sub_val
        det_val += term
        exp = superscripts.get(c, f"^{c}")
        log += (f"{indent}{matrix[0][c]:.2f} × det(\n"
            f"{pretty_matrix(minor)}\n"
            f"{indent}) × ((-1){exp} → {term:.2f}\n")

    log += sub_log
    log += f"{indent}>> det = {det_val:.2f}\n"
    return det_val, log
```

5. 추가 기능: 행렬식 계산 버튼

- `calculate_determinant_only()`: 버튼 클릭 시 행렬식만 계산하여 결과창에 출력

```
def calculate_determinant_only():
    try:
        n = len(matrix_entries)
        if n == 0:
            messagebox.showwarning("입력 오류", "먼저 Input A를 누르세요.")
            return
        matrix = get_matrix(matrix_entries)
        det, log = determinant_with_steps(matrix)
        for widget in result_frame.winfo_children():
            widget.destroy()
        frame = tk.Frame(result_frame, bg="white", bd=1, relief="solid")
        frame.pack(padx=10, pady=10, fill="both", expand=True)
        tk.Label(frame, text="Determinant Calculation Steps",
                 font=("Apple SD Gothic Neo", 10, "bold"),
                 bg="white", fg="#333").pack(pady=5)
        log_box = tk.Text(frame, height=20, width=30,
                          wrap="word", font=("Courier New", 10),
                          bg="■ #F5FBFF", fg="black",
                          insertbackground="■ #1E3A5F",
                          highlightthickness=0, borderwidth=0)
        log_box.pack(padx=10, pady=10, fill="both", expand=True)
        log_box.tag_config("explain", foreground="black")
        log_box.tag_config("matrix", foreground="■ #1565C0")
        for line in log.split("\n"):
            if any(bracket in line for bracket in ["[", "[", "|"]):
                log_box.insert("end", line + "\n", "matrix")
            else:
                log_box.insert("end", line + "\n", "explain")
        log_box.insert("end", f"\ndet(A) = {det:.2f}", "explain")
        log_box.config(state="disabled")
    except Exception as e:
        messagebox.showerror("오류", str(e))
```

- 구현 이유

: 행렬식을 이용하여 역행렬을 구하기 전에
행렬식에 대한 계산과정을 따로 확인할 수 있도록 구현

5. 추가 기능: A^T 버튼(전치행렬 출력 기능)

- 전치행렬 출력 기능

```
def transpose_matrix():  
    A = get_matrix(matrix_entries)  
    if A:  
        res = list(map(list, zip(*A)))  
        show_single_result("Transposed Matrix", res)
```

- 구현 이유

: 행렬식 기반 역행렬(Adjoint Method)을 구하는 과정에 여인자 행렬(cofactor matrix)의 전치를 취해 수반행렬 $\text{adj}(A)$ 를 만드는 단계가 필요하기 때문에, 전치행렬 연산을 별도로 구현하여 확인할 수 있도록 함

6. 프로그램 전체 구조

코드 내 역행렬 함수의 역할

- 역행렬 계산 (`calculate_inverse()`)
 - 1) 입력 행렬 불러오기 `determinant_with_steps()`
 - 행렬식(det) 계산 및 0 여부 확인
 - 2) `adjoint_inverse()`
 - 행렬식·수반행렬을 이용해 역행렬 계산
 - 3) `gauss_jordan_inverse()`
 - 가우스-조던 소거법으로 역행렬 계산
 - 4) `show_results()`
 - 두 방법의 결과 및 단계별 로그 출력
 - 5) `compare_matrices()`
 - 두 결과가 동일한지 비교 후 메시지 표시

내부 함수 관계

- `calculate_inverse()`
 - ↳ `determinant_with_steps()`
 - ↳ `adjoint_inverse()` → (`cofactor_matrix()`, `get_minor()`)
 - `determinant_with_steps()`
 - ↳ `gauss_jordan_inverse()`
 - ↳ `show_results()`
 - ↳ `compare_matrices()`

6. 프로그램 전체 구조



행렬 계산기

Matrix size(n) **Input** Inverse Determinant Input^T

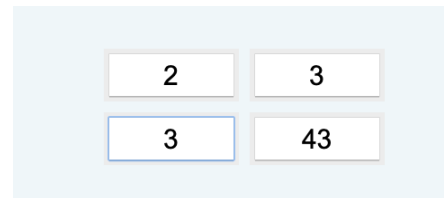
행렬 크기(n)를 입력하고 Input 버튼을 누르면 $n \times n$ 크기의 입력 칸이 자동으로 생성



행렬 계산기

Matrix size(n) 2 **Input** Inverse Determinant Input^T

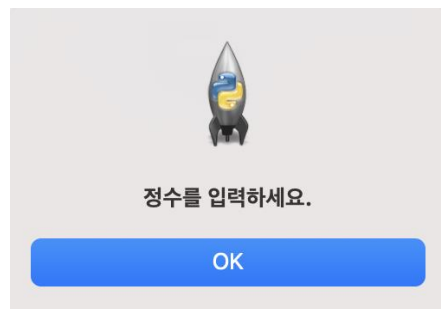
행렬 크기 결정 → 입력 칸 생성 → 행렬 원소 입력



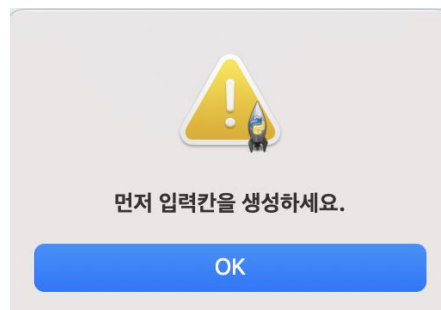
2	3
3	43

6. 프로그램 전체 구조

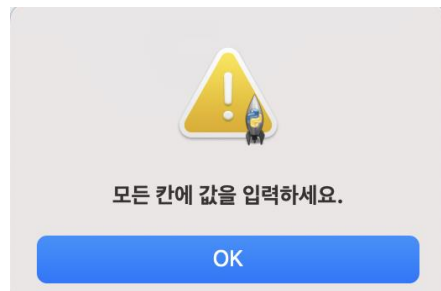
입력 시 로그



행렬 크기(n) 입력 하지 않고 Input 버튼 누르면 오류 로그 화면에 출력



행렬을 입력 하지 않고 Inverse, Determinant, Input^T 버튼 누르면 오류 로그 화면에 출력



행렬 입력칸에 수를 전부 입력하지 않고 Inverse, Determinant, Input^T 버튼 누르면 오류 로그 화면에 출력

6. 프로그램 전체 구조

Matrix size(n)

Adjoint Method (Determinant-based)

-3.00	2.00
2.00	-1.00

행렬식 det(A)
, end=
det(
[1.00 2.00]
[2.00 3.00]
)
= 1.0×3.0 - 2.0×2.0 = -1.00

여인자 행렬 (Cofactor Matrix)
[3.00 -2.00]
[-2.00 1.00]

수반행렬 adj(A) = Cofactor^T
[3.00 -2.00]
[-2.00 1.00]

A⁽⁻¹⁾ = (1/det(A)) * adj(A), det(A) = -1.00

Gauss-Jordan Elimination

-3.00	2.00
2.00	-1.00

초기 상태
[1.00 2.00] [1.00 0.00]
[2.00 3.00] [0.00 1.00]

행 0을 피벗 1.00로 나눔
[1.00 2.00] [1.00 0.00]
[2.00 3.00] [0.00 1.00]

행 1에서 2.00 × 행 0 빼기
[1.00 2.00] [1.00 0.00]
[0.00 -1.00] [-2.00 1.00]

행 1를 피벗 -1.00로 나눔
[1.00 2.00] [1.00 0.00]
[-0.00 1.00] [2.00 -1.00]

행 0에서 2.00 × 행 1 빼기
[1.00 0.00] [-3.00 2.00]

두 방법의 결과가 동일합니다.

OK

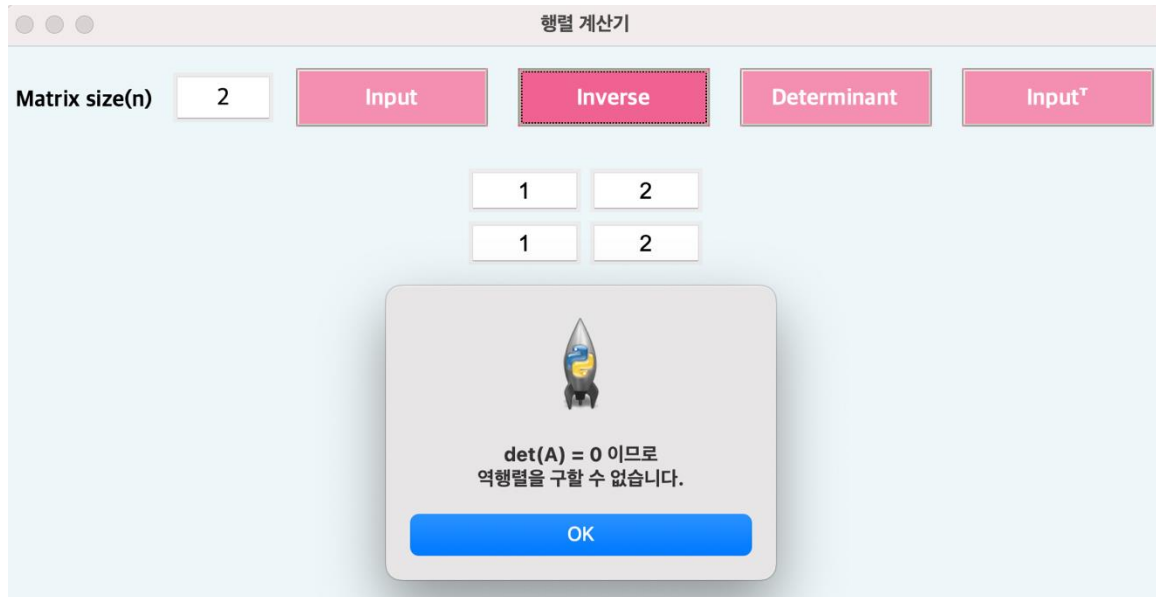
Inverse 버튼

입력한 행렬에 대한 역행렬이 계산되고 역행렬 계산 과정과 결과 출력

1) 행렬식이 0이 아닌 경우

- 행렬식 계산과 가우스-조던 소거법으로 각각 역행렬 연산 수행
- 비교 결과 로그 화면에 출력
- 역행렬 연산 과정 출력(소수 두번째 자리에서 반올림)

6. 프로그램 전체 구조



Inverse 버튼

입력한 행렬에 대한 역행렬이 계산되고 역행렬 계산 과정과 결과 출력

2) 행렬식이 0인 경우: 역행렬 구할 수 없다는 로그 화면에 출력

6. 프로그램 전체 구조

The screenshot shows a web-based matrix calculator interface. At the top, there is a label "Matrix size(n)" followed by a text input field containing the number "2". To the right of this are four pink buttons: "Input", "Inverse", "Determinant", and "Input^T". The "Determinant" button is highlighted with a dashed border. Below these buttons, there is a 2x2 grid of input fields for matrix elements. The values entered are: top-left is "1", top-right is "2", bottom-left is "2", and bottom-right is "3". In the center of the interface, there is a white box with a black border titled "Determinant Calculation Steps". Inside this box, the calculation is shown as follows:

```
det (
[ 1.00  2.00 ]
[ 2.00  3.00 ]
)
= 1.0×3.0 - 2.0×2.0 = -1.00

det (A) = -1.00
```

Determinant 버튼

입력한 행렬에 대한 행렬식 계산되고 행렬식 계산 과정과 결과 출력

6. 프로그램 전체 구조

행렬 계산기

Matrix size(n)

Transposed Matrix	
2.00	3.00
3.00	5.00

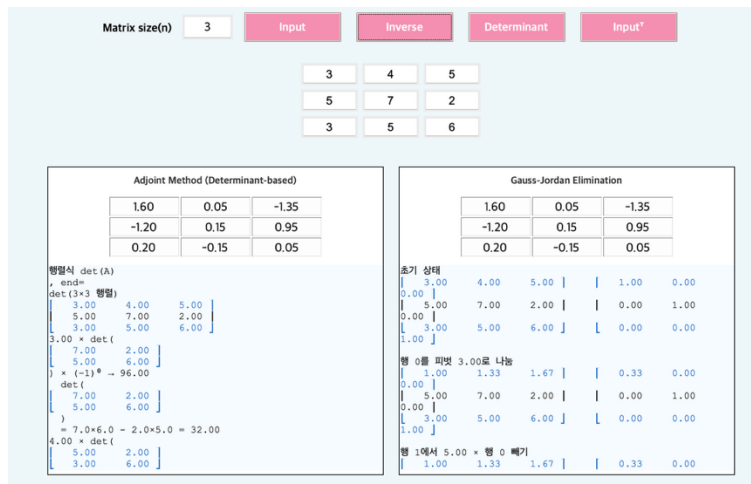
Input^T 버튼

입력한 행렬에 대한 전치행렬 계산되어 결과 출력

7. TEST(결과 정답이랑 비교)



$$= A^{-1} = \begin{bmatrix} -\frac{7}{11} & \frac{2}{11} \\ \frac{9}{11} & -\frac{1}{11} \end{bmatrix}$$



$$= A^{-1} = \frac{1}{\det(A)} C^T = \frac{1}{20} \begin{bmatrix} 32 & 7 & -27 \\ -24 & -3 & 19 \\ 4 & 3 & 1 \end{bmatrix}$$

계산된 결과

정답값

7. TEST(결과 정답이랑 비교)

Matrix size(n)

1	2	3	2
2	2	6	1
3	3	5	4
4	4	3	2

Adjoint Method (Determinant-based)

-1.00	0.09	0.40	0.16
1.00	-0.13	-0.60	0.27
-0.00	0.22	-0.00	-0.11
0.00	-0.24	0.40	-0.18

행렬식 det(A)
end=
det(4x4 행렬)

$$\begin{bmatrix} 1.00 & 2.00 & 3.00 & 2.00 \\ 2.00 & 2.00 & 6.00 & 1.00 \\ 3.00 & 3.00 & 5.00 & 4.00 \\ 4.00 & 4.00 & 3.00 & 2.00 \end{bmatrix}$$

$$1.00 \times \det \begin{bmatrix} 2.00 & 6.00 & 1.00 \\ 3.00 & 5.00 & 4.00 \\ 4.00 & 3.00 & 2.00 \end{bmatrix}$$

$$\times (-1)^0 \rightarrow 45.00$$

$$\det(3 \times 3 \text{ 행렬})$$

$$\begin{bmatrix} 2.00 & 6.00 & 1.00 \\ 3.00 & 5.00 & 4.00 \\ 4.00 & 3.00 & 2.00 \end{bmatrix}$$

$$2.00 \times \det \begin{bmatrix} 5.00 & 4.00 \end{bmatrix}$$

Gauss-Jordan Elimination

-1.00	0.09	0.40	0.16
1.00	-0.13	-0.60	0.27
-0.00	0.22	0.00	-0.11
-0.00	-0.24	0.40	-0.18

초기 상태

$$\left[\begin{array}{cccc|ccc} 1.00 & 2.00 & 3.00 & 2.00 & 2.00 & 1.00 & 1.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 1.00 & 0.00 & 0.00 \\ 2.00 & 2.00 & 6.00 & 6.00 & 1.00 & 0.00 & 0.00 \\ 1.00 & 0.00 & 0.00 & 0.00 & 3.00 & 5.00 & 4.00 \\ 3.00 & 3.00 & 5.00 & 4.00 & 0.00 & 1.00 & 0.00 \\ 0.00 & 1.00 & 0.00 & 0.00 & 4.00 & 3.00 & 2.00 \\ 4.00 & 4.00 & 3.00 & 2.00 & 0.00 & 0.00 & 1.00 \end{array} \right]$$

 행 0를 피벗 1.00로 나눔

$$\left[\begin{array}{cccc|ccc} 1.00 & 2.00 & 3.00 & 2.00 & 2.00 & 1.00 & 1.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 1.00 & 0.00 & 0.00 \\ 2.00 & 2.00 & 6.00 & 6.00 & 1.00 & 0.00 & 0.00 \\ 1.00 & 0.00 & 0.00 & 0.00 & 3.00 & 5.00 & 4.00 \\ 3.00 & 3.00 & 5.00 & 4.00 & 0.00 & 1.00 & 0.00 \\ 0.00 & 1.00 & 0.00 & 0.00 & 4.00 & 3.00 & 2.00 \\ 4.00 & 4.00 & 3.00 & 2.00 & 0.00 & 0.00 & 1.00 \end{array} \right]$$

=

$$A^{-1} = \begin{bmatrix} -1 & \frac{4}{45} & \frac{2}{5} & \frac{7}{45} \\ 1 & -\frac{2}{15} & -\frac{3}{5} & \frac{4}{15} \\ 0 & \frac{2}{9} & 0 & -\frac{1}{9} \\ 0 & -\frac{11}{45} & \frac{2}{5} & -\frac{8}{45} \end{bmatrix}$$

계산된 결과

정답값